Prática — Mural da Turma com Flutter e AWS

Oct 14, 2026 · @Guilherme Rodrigues

Cada dupla vai criar, do zero, um app Flutter que lê e grava recados numa API real na AWS. A API é uma só para a turma inteira: o recado que vocês postarem aparece no app de todo mundo.

O que vocês vão construir

Um mural de recados: a pessoa digita o nome e uma mensagem, envia, e vê a lista de recados da turma inteira. O visual é livre.

Trabalhem por etapas. Cada nível só começa quando o anterior funciona.

Etapa	O app precisa	Rota usada
1 · Ler	Abrir e mostrar a lista de recados (autor e texto), com indicador de carregando e mensagem de erro	`GET /recados`
2 · Escrever	Ter um formulário com nome e recado; ao enviar, o recado aparece na lista	`POST /recados`
3 · Apagar	Ter um botão em cada recado que o remove da lista	`DELETE /recados/{id}`
4 · Curtir	Ter um botão de curtir com o número de curtidas de cada recado	`POST /recados/{id}/curtir`

Requisitos

- **Gerenciamento de estado com BLoC** (pacote `flutter_bloc`). A tela só dispara eventos e desenha estados; ela nunca chama a API diretamente.
- **Requisições HTTP** com o pacote `http`.
- **Organização de `lib/`** em quatro pastas:

Pasta	Responsabilidade
`models/`	A classe do recado e a conversão de JSON para objeto

Pasta	Responsabilidade
`provider/`	A única parte que fala com a internet: um método por rota da API
`bloc/`	Eventos, estado e o BLoC que liga a tela ao provider
`view/`	Telas e widgets

O `main.dart` fica na raiz de `lib/` e só monta o app e o BLoC.

A API

Rotas

Método e caminho	Corpo enviado	Sucesso	Resposta
`GET /recados`	—	`200`	Lista de recados, os mais recentes primeiro (até 300)
`POST /recados`	`{"autor": "...", "texto": "..."}`	`201`	O recado criado
`DELETE /recados/{id}`	—	`204`	Vazia
`POST /recados/{id}/curtir`	—	`200`	O recado com as curtidas atualizadas

No `POST /recados`, envie o cabeçalho `Content-Type: application/json`.

Formato de um recado

```
{
  "id": "3f2a9c1e-5b7d-4e8a-9c21-7d4e5f6a8b90",
  "autor": "Ana",
  "texto": "Olá, nuvem!",
  "curtidas": 0,
  "criadoEm": "2026-10-14T19:30:00+00:00"
}
```

- O app envia só `autor` e `texto`. A nuvem gera `id`, `curtidas` e `criadoEm`.
- `id` é texto, não número.
- `curtidas` é inteiro.

- `criadoEm` é data no formato ISO 8601, em UTC.

Regras e erros

Status	Quando acontece
`400`	`autor` vazio ou com mais de 40 caracteres; `texto` vazio ou com mais de 280; JSON inválido
`404`	Rota que não existe, ou curtir um recado que já foi apagado
`429`	Requisições demais em pouco tempo (veja as dicas)
`500`	Erro no servidor: avise o professor

Corpo de erro: `{"erro": "mensagem"}`. Validem os limites também no app, para o usuário não ver um 400.

Como começar

```
flutter create mural_turma --platforms=web,android
cd mural_turma
flutter pub add flutter_bloc http
flutter run -d chrome
```

Uma ordem que funciona:

1. **models:** a classe `Recado` com um construtor `fromJson`.
2. **provider:** a URL base e o método `listar()`. Testem chamando-o direto no `main` com `print`, antes de existir qualquer tela.
3. **bloc:** os eventos, a classe de estado (lista, carregando e erro) e o BLoC com o handler de carregar.
4. **view:** a tela com `BlocBuilder` mostrando os três estados. Fechou o nível 1.
5. Repitam o ciclo provider → bloc → view para criar, apagar e curtir.

Dicas e erros comuns

Sintoma	Causa e solução
Acentos aparecem como `OlÃ¡`	Decodifiquem com `utf8.decode(resposta.bodyBytes)`, não com `resposta.body`.
`404` em tudo	Barra no final da URL base, ou caminho escrito errado (`/recado`).
`400` ao enviar	Faltou o cabeçalho `Content-Type`, o corpo não passou por `jsonEncode`, ou as chaves não são exatamente `autor` e `texto`.
`type 'Null' is not a subtype of type 'String'`	O `fromJson` lê uma chave que não existe. Confiram os nomes no formato acima.
`type 'int' is not a subtype of type 'String'`	`curtidas` é número; leiam como `int`.
O recado foi criado mas não aparece	Recarreguem a lista depois do `POST`, e usem `await` antes de recarregar.
`429` para a turma inteira	Algum app está chamando a API sem parar. **Nunca chamem a API dentro do `build()`**: dispare o evento de carregar uma vez, ao criar o BLoC.
`Failed to fetch`	URL errada ou sem internet. Se a URL estiver certa, chamem o professor.

Outras dicas:

- O `GET` já vem ordenado do mais novo para o mais antigo. Não precisam ordenar.
- Para atualizar a lista com recados de outras duplas, coloquem um botão de recarregar. Se fizerem atualização automática, usem no mínimo 10 segundos de intervalo.
- Abram a URL base com `/recados` no navegador

Entrega

1. **Na aula:** mostrem ao professor